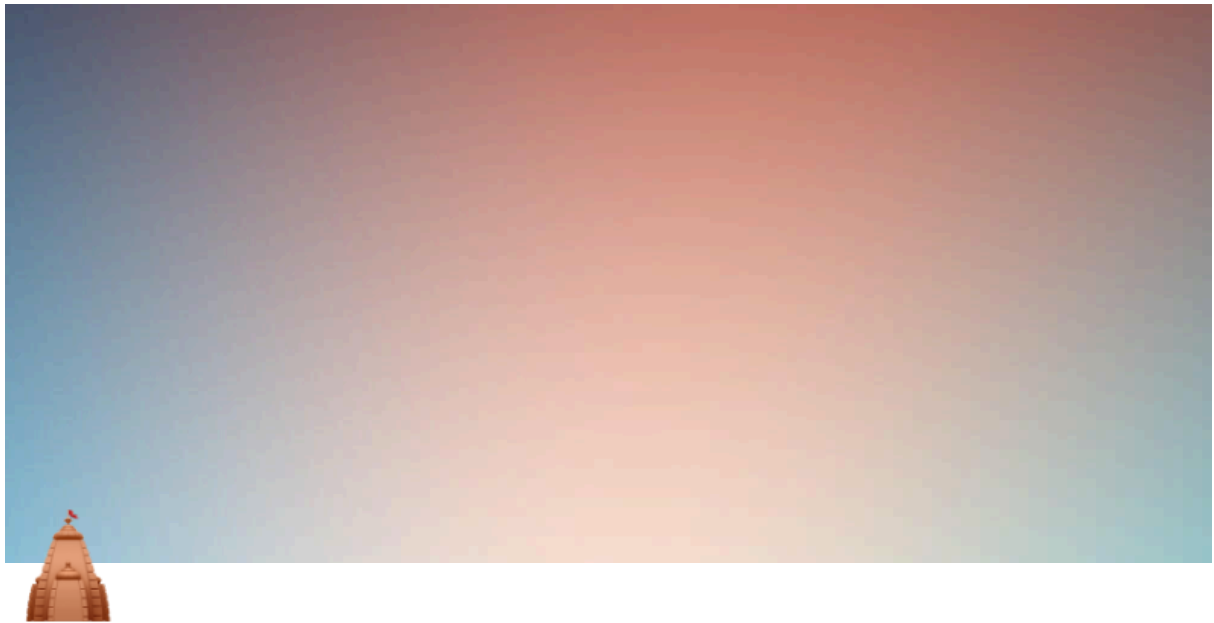




238. 除自身以外数组的乘积

给你一个整数数组 `nums`，返回 数组 `answer`，其中 `answer[i]` 等于 `nums` 中除 `nums[i]` 之外其余各元素的乘积。

题目数据 **保证** 数组 `nums` 之中任意元素的全部前缀元素和后缀的乘积都在 **32 位** 整数范围内。

请 **不要使用除法**，且在 `O(n)` 时间复杂度内完成此题。

示例 1:

输入: `nums = [1,2,3,4]` 输出: `[24,12,8,6]`

示例 2:

输入: `nums = [-1,1,0,-3,3]`
输出: `[0,0,9,0,0]`

提示:

- `2 <= nums.length <= 105`
- `30 <= nums[i] <= 30`
- 输入 **保证** 数组 `answer[i]` 在 **32 位** 整数范围内

进阶：你可以在 `O(1)` 的额外空间复杂度内完成这个题目吗？（出于对空间复杂度分析的目的，输出数组 **不被视为** 额外空间。）

2. 解析

好的，我们来分析“除自身以外数组的乘积”这个问题 (LeetCode 238)。

1. 问题理解

- **目标：** 给定一个整数数组 `nums`，要求返回一个新的数组 `answer`，其中 `answer[i]` 等于 `nums` 数组中除了 `nums[i]` 之外所有元素的乘积。
- **约束：**
 - 算法时间复杂度必须是 $O(n)$ 。
 - 不能使用除法。
 - 空间复杂度要求 $O(1)$ 的额外空间（输出数组 `answer` 不算在额外空间内）。

2. 你的初步想法（普通人思路 - 尝试直接计算）

你的代码片段 `res[i]=nums[i+1]*nums[i+2]` 看起来是想直接为 `res[i]` 计算其他元素的乘积，但这种方式有几个问题：

- **不完整：** 它只考虑了 `i+1` 和 `i+2` 的元素，没有包含 `i` 之前以及 `i+2` 之后的所有其他元素。
- **越界：** 当 `i` 接近数组末尾时，`nums[i+1]` 或 `nums[i+2]` 会导致数组索引越界错误。
- **变量类型：** `res` 应该是一个数组 `int[]`，而不是 `int`。

完善你的初步想法（暴力解法 - $O(n^2)$ 时间）

基于“计算除自身外所有元素的乘积”这个直接想法，我们可以这样做：

1. 创建一个结果数组 `answer`，大小与 `nums` 相同。

2. 遍历 `nums` 数组，对于每个索引 `i`（从 0 到 $n-1$ ）：

a. 初始化一个变量

`product = 1`。

b. 再次遍历

`nums` 数组，对于每个索引 `j`（从 0 到 $n-1$ ）：

i.

如果 `i!=j`（即，不是当前元素自身），则将 `nums[j]` 乘入 `product`：`product = product *`

`nums[j]`。

c. 内层循环结束后，

`product` 就是除了 `nums[i]` 之外所有元素的乘积。将其存入结果数组：`answer[i] = product`。

3. 返回 `answer` 数组。

缺点：这个方法需要两层嵌套循环，时间复杂度是 $O(n^2)$ ，不满足题目 $O(n)$ 的要求。但它满足了“不使用除法”的要求。

3. 更好的方法（前缀积与后缀积 - $O(n)$ 时间）

为了达到 $O(n)$ 的时间复杂度，并且不使用除法，我们可以利用前缀积和后缀积。

`answer[i]` 的值，其实等于 `(nums[0] * ... * nums[i-1]) * (nums[i+1] * ... * nums[n-1])`。
也就是

`i` 左边所有元素的乘积 乘以 `i` 右边所有元素的乘积。

我们可以分两步计算：

- **第一步（计算前缀积）：** 创建一个结果数组 `answer`。让 `answer[i]` 先存储 `nums[0]` 到 `nums[i-1]` 的乘积（即 `i` 左侧所有元素的乘积）。我们可以通过一次从左到右的遍历来完成。
 - `answer[0]` 左边没有元素，所以其前缀积可以看作 1。
 - 对于 `i > 0`，`answer[i] = nums[0] * ... * nums[i-1] = answer[i-1] * nums[i-1]`。
- **第二步（计算后缀积并合并）：** 现在 `answer[i]` 存储了左侧的乘积。我们还需要乘以右侧的乘积。我们可以通过一次从右到左的遍历来完成，同时维护一个变量 `rightProduct` 表示当前位置 `i` 右侧所有元素的乘积。
 - 从右向左遍历 `i`（从 `n-1` 到 0）。
 - 对于当前的 `i`，`answer[i]`（它现在存的是左侧乘积）需要再乘以当前的 `rightProduct`（右侧乘积），得到最终结果：`answer[i] = answer[i] * rightProduct`。
 - 然后，更新 `rightProduct`，为下一次迭代（即 `i-1` 位置）做准备，将当前元素 `nums[i]` 乘进去：`rightProduct = rightProduct * nums[i]`。
 - 初始时（当 `i = n-1` 时），`rightProduct` 应该为 1，因为 `n-1` 右侧没有元素。

复杂度：

- 时间复杂度： $O(n)$ 。我们进行了两次独立的线性遍历。
- 空间复杂度： $O(1)$ 。我们只使用了结果数组 `answer` 和一个额外的变量 `rightProduct`，满足 $O(1)$ 额外空间要求。

4. 示例演示（前缀积与后缀积法）

`nums = [1, 2, 3, 4]`

1. 初始化: `answer = [?, ?, ?, ?]`

2. 第一遍（计算前缀积，存入 `answer`）:

- `i = 0`: `answer[0] = 1` (左侧没有元素)
- `i = 1`: `answer[1] = answer[0] * nums[0] = 1 * 1 = 1`
- `i = 2`: `answer[2] = answer[1] * nums[1] = 1 * 2 = 2`
- `i = 3`: `answer[3] = answer[2] * nums[2] = 2 * 3 = 6`
- 此时 `answer = [1, 1, 2, 6]` (`answer[i]` 存储了 `nums[0...i-1]` 的乘积)

3. 第二遍（计算后缀积并合并）:

- 初始化 `rightProduct = 1`。
- `i = 3` (从右往左):
 - `answer[3] = answer[3] * rightProduct = 6 * 1 = 6`
 - 更新 `rightProduct = rightProduct * nums[3] = 1 * 4 = 4`
- `i = 2`:
 - `answer[2] = answer[2] * rightProduct = 2 * 4 = 8`
 - 更新 `rightProduct = rightProduct * nums[2] = 4 * 3 = 12`
- `i = 1`:
 - `answer[1] = answer[1] * rightProduct = 1 * 12 = 12`
 - 更新 `rightProduct = rightProduct * nums[1] = 12 * 2 = 24`
- `i = 0`:
 - `answer[0] = answer[0] * rightProduct = 1 * 24 = 24`
 - 更新 `rightProduct = rightProduct * nums[0] = 24 * 1 = 24`

4. 完成! 最终 `answer = [24, 12, 8, 6]`。

5. 代码实现（前缀积与后缀积法）

```
/*
 * @lc app=leetcode.cn id=238 lang=java
 * @lcpr version=30204
```

```

*
* [238] 除自身以外数组的乘积
*/

// @lcpr-template-start

// @lcpr-template-end
// @lc code=start
class Solution {
    public int[] productExceptSelf(int[] nums) {
        // 1. 边界处理
        if (nums == null || nums.length == 0) {
            return new int[0]; // 或者根据需要返回 null 或抛出异常
        }

        int n = nums.length;
        int[] answer = new int[n];

        // 2. 第一遍：计算前缀积（不包括当前元素）并存入 answer 数组
        // answer[i] 将存储 nums[0] * ... * nums[i-1]
        answer[0] = 1; // 第一个元素左边没有元素，前缀积为 1
        for (int i = 1; i < n; i++) {
            answer[i] = answer[i - 1] * nums[i - 1];
        }

        // 3. 第二遍：计算后缀积并与前缀积合并
        int rightProduct = 1; // 用于存储当前位置右侧所有元素的乘积
        for (int i = n - 1; i >= 0; i--) {
            // 对于索引 i，answer[i] 已经包含了左侧的乘积
            // 现在需要乘以右侧的乘积 rightProduct
            answer[i] = answer[i] * rightProduct;
            // 更新 rightProduct，为下一个位置 (i-1) 做准备
            rightProduct *= nums[i]; // 将当前元素乘入后缀积
        }

        return answer;
    }
}

```

```

}
// @lc code=end

/*
// @lcpr case=start
// [1,2,3,4]\\n
// @lcpr case=end

// @lcpr case=start
// [-1,1,0,-3,3]\\n
// @lcpr case=end

*/

```

6. 如何在面试中讲解

1. **确认问题：** "好的，目标是计算一个数组，每个位置的值是原数组除自身外所有元素的乘积，要求 $O(n)$ 时间， $O(1)$ 额外空间，且不能用除法。 "
2. **排除直接方法：** "如果允许用除法，可以先计算总乘积，然后每个位置用总乘积除以当前元素。但题目禁止除法。如果用嵌套循环暴力计算，时间是 $O(n^2)$ ，不满足要求。 "
3. **核心思路 - 前后缀积：** "我们可以观察到， `answer[i]` 等于 `i` 左边所有元素的乘积乘以 `i` 右边所有元素的乘积。这提示我们可以分两步计算。 "
4. **两趟遍历：** "我们可以用两趟遍历来解决：
 - **第一趟（从左到右）：** 计算每个位置 `i` 的**前缀积**（即 `nums[0]` 到 `nums[i-1]` 的乘积），并将这个结果先存放在 `answer[i]` 中。 `answer[0]` 的前缀积是 1。
 - **第二趟（从右到左）：** 维护一个变量 `rightProduct`，表示当前位置 `i` 右侧所有元素的乘积。在遍历时，用 `answer[i]`（当前存储的是前缀积）乘以 `rightProduct`，得到最终结果并更新回 `answer[i]`。同时，更新 `rightProduct` 供下一次迭代使用（将 `nums[i]` 乘进去）。 "
5. **复杂度：** "这样做了两趟独立的 $O(n)$ 遍历，总时间复杂度是 $O(n)$ 。我们只用了结果数组和一个额外变量 `rightProduct`，所以额外空间复杂度是 $O(1)$ ，满足所有要求。 "
6. **示例（可选）：** "可以用 `[1, 2, 3, 4]` 这个例子演示一下两趟遍历的过程..."

这样讲解清晰地展示了问题的难点、思考过程以及最优解的原理和实现细节。

找到具有 1 个许可证类型的类似代码